

Laporan Baseline Tahap 0

Dokumen ini adalah artefak awal untuk Tahap 0 - Audit Baseline dan Reproducibility.

Tujuan

Tahap 0 bertujuan memastikan baseline CDE-Mapper dapat dijalankan ulang secara konsisten sebelum adaptasi Bahasa Indonesia dilakukan. Fokus tahap ini adalah audit struktur, konfigurasi, dependency, input smoke test, dan kesiapan eksekusi.

Artefak yang Dibuat

- `configs/baseline.yaml`: konfigurasi deklaratif baseline dan command inference.
- `data/input/baseline_smoke.csv`: input kecil untuk smoke test loader custom.
- `scripts/baseline_smoke.py`: audit lokal tanpa network/LLM untuk import modul, loader, JSON template, dan reservoir SQLite sementara.
- `scripts/audit_baseline_wsl.sh`: wrapper WSL yang menjalankan audit dengan conda env `cde-mapper`.
- `Riset/LAPORAN_BASELINE.md`: laporan audit tahap 0.

Environment

Sesuai permintaan, audit ditargetkan berjalan di WSL dengan conda environment:

```
conda activate cde-mapper
```

File `environment.yml` sengaja tidak digunakan.

Command wrapper:

```
wsl -e bash -lc "cd /mnt/d/Program/cde_mapper && bash  
scripts/audit_baseline_wsl.sh"
```

Jika `conda` tidak otomatis tersedia pada shell WSL non-interaktif, jalankan dari terminal WSL:

```
cd /mnt/d/Program/cde_mapper  
source ~/miniconda3/etc/profile.d/conda.sh  
conda activate cde-mapper  
python scripts/baseline_smoke.py
```

Baseline Runner Saat Ini

Entry point utama baseline:

```
python run.py
```

Command inference minimal untuk data custom:

```
python run.py \  
  --flag inference \  
  --input_file data/input/baseline_smoke.csv \  
  --custom_data \  
  --is_omop_data \  
  --collection_name concept_mapping_1 \  
  --llm_id google/gemma-3n-E4B-it \  
  --topk 5 \  
  --output_file data/output/baseline_smoke_mapped.csv
```

Catatan: command inference penuh membutuhkan koneksi ke Qdrant/Athena dan runtime/kredensial LLM yang sesuai. Smoke test Tahap 0 tidak memanggil network atau LLM.

Dependency Aktual

Sumber dependency yang ditemukan:

- `pyproject.toml`: dependency proyek Poetry, termasuk `langchain`, `qdrant-client`, `fastembed`, `torch`, `transformers`, `pandas`, `rapidfuzz`, dan provider LLM.
- `requirements.txt`: lock/hasil pip-compile yang lebih panjang dan tampaknya berasal dari baseline upstream.

Catatan audit:

- `README.md` masih menyebut `requirements.in`, tetapi file yang tersedia di repo saat ini adalah `requirements.txt` dan `pyproject.toml`.
- Untuk reproducibility penelitian, dependency sebaiknya distandarkan ke satu sumber utama. Karena user meminta memakai conda env `cde-mapper` di WSL, status paket aktual perlu diambil dari environment tersebut saat audit dijalankan.

Smoke Test

Smoke test lokal memeriksa:

- import modul baseline inti;
- loader custom CSV terhadap `data/input/baseline_smoke.csv`;
- validitas JSON `data/input/mapping_templates.json`;
- inisialisasi reservoir SQLite sementara dari `database_data`;
- import stack retrieval tanpa memanggil Qdrant/Athena/LLM.

Hasil audit otomatis akan ditulis ke:

```
Riset/baseline_audit_result.json
```

Hasil eksekusi pada WSL conda env `cde-mapper`:

- status: berhasil;
- Python: `3.10.19` dari conda-forge;
- platform: WSL2 Linux;
- durasi audit: `3.45` detik;
- jumlah query smoke yang berhasil dimuat: `3`;
- jumlah row reservoir sementara: `10`;
- `data/input/mapping_templates.json` valid dan memiliki `database_data=2063`;
- retrieval stack penuh tidak dipanggil pada smoke test untuk menghindari kebutuhan Qdrant, Athena, GPU, dan LLM.

Temuan Awal

- Baseline sudah memiliki entry point `run.py`, tetapi command di README lama masih mengarah ke path `mapping_tool/rag/vector_index.py`; command Tahap 0 mengarahkan kembali ke `run.py`.
- `rag/data_loader.py` memiliki loader custom CSV yang cocok untuk data dictionary lokal dengan kolom `variablename`, `variablelabel`, `categorical`, `units`, `formula`, dan `visits`.
- Reservoir lokal baseline dikelola dengan SQLite melalui `rag/sql.py` dan konfigurasi `DB_FILE = 'variables.db'`.
- File `variables.db` tidak perlu ada sebelum run; database dibuat otomatis saat `DataManager` dijalankan.
- Smoke test sengaja memakai temporary SQLite database agar tidak mengubah reservoir kerja.
- `rag/__init__.py` sebelumnya melakukan wildcard import ke hampir seluruh stack retrieval/LLM. Ini membuat import ringan gagal ketika CUDA device atau API key LLM tidak tersedia. File tersebut sudah dibuat ringan sehingga modul spesifik dapat diimpor secara reproducible.
- Audit dipaksa CPU-only dengan `CUDA_VISIBLE_DEVICES=""`, karena Tahap 0 tidak membutuhkan GPU.

Gap Teknis untuk Tahap Berikutnya

- Standardisasi dependency perlu diputuskan: `pyproject.toml`, `requirements.txt`, atau export dari conda env `cde-mapper`.
- Command inference penuh masih tergantung koneksi Qdrant/Athena dan LLM.
- Belum ada dataset gold Bahasa Indonesia untuk evaluasi mapping final.
- README perlu dipertahankan sinkron dengan entry point aktual jika pipeline berubah.

Implementasi Full Inference dan Evaluasi

Tahap 0 dilengkapi dengan artefak berikut:

- `scripts/audit_baseline_integration.py`: validasi online Qdrant, Athena, dan Together;
- `scripts/baseline_experiment.py`: retrieval dense dan sparse secara berurutan, merger Top-10, latency per sumber, error per query, serta manifest run;
- `scripts/baseline_llm_rerank.py`: reranking kandidat tersimpan menggunakan `google/gemma-3n-E4B-it`;

- `evaluation/baseline_retrieval_eval.py`: accuracy, precision, recall, MRR, NDCG, coverage, error rate, serta distribusi latency;
- `scripts/capture_baseline_environment.py`: snapshot Python, platform, commit, conda package, dan pip package tanpa secret;
- `data/gold/baseline_retrieval_gold.jsonl`: gold subset 10 query dengan OMOP ID dan vocabulary code ekuivalen.

Command reproducible:

```
conda run --no-capture-output -n cde-mapper \  
python scripts/baseline_experiment.py --run-id baseline-gemma3n-stage0 --skip-llm-check  
  
conda run --no-capture-output -n cde-mapper \  
python scripts/baseline_llm_rerank.py \  
--input data/output/baseline/baseline-gemma3n-stage0.jsonl \  
--output data/output/baseline/baseline-gemma3n-stage0-reranked.jsonl  
  
conda run --no-capture-output -n cde-mapper \  
python evaluation/baseline_retrieval_eval.py \  
--input data/output/baseline/baseline-gemma3n-stage0-reranked.jsonl \  
--output Riset/baseline_reranked_metrics.json
```

`--skip-llm-check` pada retrieval run hanya mencegah model LLM hidup bersamaan dengan model embedding. Validasi LLM online dan reranking aktual tetap dijalankan sebagai proses terpisah.

Hasil Baseline 10 Juli 2026

Retrieval hybrid Qdrant dense-sparse:

Metrik	Nilai
Coverage	1.00
Accuracy@1 / @3 / @5 / @10	0.60 / 0.60 / 0.70 / 0.90
Precision@1 / @3 / @5 / @10	0.60 / 0.20 / 0.14 / 0.09
Recall@1 / @3 / @5 / @10	0.60 / 0.60 / 0.70 / 0.90
MRR	0.64
NDCG@10	0.6965
Latency mean / p50 / p95	1488.18 / 1386.53 / 1959.57 ms

Setelah reranking Gemma:

Metrik	Nilai
Accuracy@1 / @3 / @5 / @10	0.40 / 0.70 / 0.80 / 0.90

Metrik	Nilai
MRR	0.5843
NDCG@10	0.6613
Reranking berhasil	10/10 query

Gemma memperbaiki cakupan ranking @3 dan @5, tetapi menurunkan Accuracy@1 dan MRR pada subset ini. Karena itu reranking belum boleh diasumsikan selalu meningkatkan hasil dan perlu prompt/calibration study pada tahap berikutnya.

Audit integrasi mengonfirmasi collection Qdrant `concept_mapping_1` tersedia dengan 3.695.703 point dan model Gemma dapat dipanggil melalui `TOGETHER_API_KEY`. Athena mengembalikan HTTP 403 untuk seluruh query. `partial_source_error_rate=1.0` mencatat gangguan tersebut, sedangkan `query_failure_rate=0.0` karena Qdrant tetap menghasilkan kandidat untuk semua query.

Status

Status: Tahap 0 selesai sebagai baseline reproducible dengan smoke audit, integration audit, dependency snapshot, output Top-k, manifest, full Gemma reranking, metrik, coverage, dan latency. Athena berstatus keterbatasan eksternal terdokumentasi; eksperimen memakai fallback Qdrant dense-sparse dan tidak menyamakan sumber yang gagal sebagai passed.